

The FFT method employs a divide-and-conquer strategy, using the even-indexed and odd-indexed coefficients of $A(x)$ separately to define the two new polynomials $A^{\text{even}}(x)$ and $A^{\text{odd}}(x)$ of degree-bound $n/2$:

$$\begin{aligned} A^{\text{even}}(x) &= a_0 + a_2x + a_4x^2 + \cdots + a_{n-2}x^{n/2-1}, \\ A^{\text{odd}}(x) &= a_1 + a_3x + a_5x^2 + \cdots + a_{n-1}x^{n/2-1}. \end{aligned}$$

Note that A^{even} contains all the even-indexed coefficients of A (the binary representation of the index ends in 0) and A^{odd} contains all the odd-indexed coefficients (the binary representation of the index ends in 1). It follows that

$$A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2), \quad (30.9)$$

so that the problem of evaluating $A(x)$ at $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ reduces to

1. evaluating the degree-bound $n/2$ polynomials $A^{\text{even}}(x)$ and $A^{\text{odd}}(x)$ at the points

$$(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2, \quad (30.10)$$

and then

2. combining the results according to equation (30.9).

By the halving lemma, the list of values (30.10) consists not of n distinct values but only of the $n/2$ complex $(n/2)$ th roots of unity, with each root occurring exactly twice. Therefore, the FFT recursively evaluates the polynomials A^{even} and A^{odd} of degree-bound $n/2$ at the $n/2$ complex $(n/2)$ th roots of unity. These subproblems have exactly the same form as the original problem, but are half the size, dividing an n -element DFT_n computation into two $n/2$ -element $\text{DFT}_{n/2}$ computations. This decomposition is the basis for the FFT procedure on the next page, which computes the DFT of an n -element vector $a = (a_0, a_1, \dots, a_{n-1})$, where n is an exact power of 2.

The FFT procedure works as follows. Lines 1–2 represent the base case of the recursion. The DFT of 1 element is the element itself, since in this case

$$\begin{aligned} y_0 &= a_0 \omega_1^0 \\ &= a_0 \cdot 1 \\ &= a_0. \end{aligned}$$

Lines 5–6 define the coefficient vectors for the polynomials A^{even} and A^{odd} . Lines 3, 4, and 12 guarantee that ω is updated properly so that whenever lines 10–11 are executed, $\omega = \omega_n^k$. (Keeping a running value of ω from iteration to iteration saves

```

FFT( $a, n$ )
1  if  $n == 1$ 
2      return  $a$                                 // DFT of 1 element is the element itself
3   $\omega_n = e^{2\pi i/n}$ 
4   $\omega = 1$ 
5   $a^{\text{even}} = (a_0, a_2, \dots, a_{n-2})$ 
6   $a^{\text{odd}} = (a_1, a_3, \dots, a_{n-1})$ 
7   $y^{\text{even}} = \text{FFT}(a^{\text{even}}, n/2)$ 
8   $y^{\text{odd}} = \text{FFT}(a^{\text{odd}}, n/2)$ 
9  for  $k = 0$  to  $n/2 - 1$                         // at this point,  $\omega = \omega_n^k$ 
10      $y_k = y_k^{\text{even}} + \omega y_k^{\text{odd}}$ 
11      $y_{k+(n/2)} = y_k^{\text{even}} - \omega y_k^{\text{odd}}$ 
12      $\omega = \omega \omega_n$ 
13 return  $y$ 

```

time over computing ω_n^k from scratch each time through the **for** loop.⁴) Lines 7–8 perform the recursive $\text{DFT}_{n/2}$ computations, setting, for $k = 0, 1, \dots, n/2 - 1$,

$$y_k^{\text{even}} = A^{\text{even}}(\omega_{n/2}^k),$$

$$y_k^{\text{odd}} = A^{\text{odd}}(\omega_{n/2}^k),$$

or, since $\omega_{n/2}^k = \omega_n^{2k}$ by the cancellation lemma,

$$y_k^{\text{even}} = A^{\text{even}}(\omega_n^{2k}),$$

$$y_k^{\text{odd}} = A^{\text{odd}}(\omega_n^{2k}).$$

Lines 10–11 combine the results of the recursive $\text{DFT}_{n/2}$ calculations. For the first $n/2$ results $y_0, y_1, \dots, y_{n/2-1}$, line 10 yields

$$\begin{aligned}
 y_k &= y_k^{\text{even}} + \omega_n^k y_k^{\text{odd}} \\
 &= A^{\text{even}}(\omega_n^{2k}) + \omega_n^k A^{\text{odd}}(\omega_n^{2k}) \\
 &= A(\omega_n^k) \quad \text{(by equation (30.9))} .
 \end{aligned}$$

For $y_{n/2}, y_{n/2+1}, \dots, y_{n-1}$, letting $k = 0, 1, \dots, n/2 - 1$, line 11 yields

⁴ The downside of iteratively updating ω is that round-off errors can accumulate, especially for larger input sizes. Several techniques to limit the magnitude of FFT round-off errors have been proposed, but are beyond the scope of this book. If several FFTs are going to be run on inputs of the same size, then it might be worthwhile to directly precompute a table of all $n/2$ values of ω_n^k .

$$\begin{aligned}
y_{k+(n/2)} &= y_k^{\text{even}} - \omega_n^k y_k^{\text{odd}} \\
&= y_k^{\text{even}} + \omega_n^{k+(n/2)} y_k^{\text{odd}} && (\text{since } \omega_n^{k+(n/2)} = -\omega_n^k) \\
&= A^{\text{even}}(\omega_n^{2k}) + \omega_n^{k+(n/2)} A^{\text{odd}}(\omega_n^{2k}) \\
&= A^{\text{even}}(\omega_n^{2k+n}) + \omega_n^{k+(n/2)} A^{\text{odd}}(\omega_n^{2k+n}) && (\text{since } \omega_n^{2k+n} = \omega_n^{2k}) \\
&= A(\omega_n^{k+(n/2)}) && (\text{by equation (30.9)}) .
\end{aligned}$$

Thus the vector y returned by FFT is indeed the DFT of the input vector a .

Lines 10 and 11 multiply each value y_k^{odd} by ω_n^k , for $k = 0, 1, \dots, n/2 - 1$. Line 10 adds this product to y_k^{even} , and line 11 subtracts it. Because each factor ω_n^k appears in both its positive and negative forms, we call the factors ω_n^k **twiddle factors**.

To determine the running time of the procedure FFT, note that exclusive of the recursive calls, each invocation takes $\Theta(n)$ time, where n is the length of the input vector. The recurrence for the running time is therefore

$$\begin{aligned}
T(n) &= 2T(n/2) + \Theta(n) \\
&= \Theta(n \lg n) ,
\end{aligned}$$

by case 2 of the master theorem (Theorem 4.1). Thus the FFT can evaluate a polynomial of degree-bound n at the complex n th roots of unity in $\Theta(n \lg n)$ time.

Interpolation at the complex roots of unity

The polynomial multiplication scheme entails converting from coefficient form to point-value form by evaluating the polynomial at the complex roots of unity, point-wise multiplying, and finally converting from point-value form back to coefficient form by interpolating. We've just seen how to evaluate, so now we'll see how to interpolate the complex roots of unity by a polynomial. To interpolate, we'll write the DFT as a matrix equation and then look at the form of the matrix inverse.

From equation (30.4), we can write the DFT as the matrix product $y = V_n a$, where V_n is a Vandermonde matrix containing the appropriate powers of ω_n :

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} .$$

The (k, j) entry of V_n is ω_n^{kj} , for $j, k = 0, 1, \dots, n - 1$. The exponents of the entries of V_n form a multiplication table for factors 0 to $n - 1$.

For the inverse operation, which we write as $a = \text{DFT}_n^{-1}(y)$, multiply y by the matrix V_n^{-1} , the inverse of V_n .

Theorem 30.7

For $j, k = 0, 1, \dots, n-1$, the (j, k) entry of V_n^{-1} is ω_n^{-jk}/n .

Proof We show that $V_n^{-1}V_n = I_n$, the $n \times n$ identity matrix. Consider the (k, k') entry of $V_n^{-1}V_n$:

$$\begin{aligned} [V_n^{-1}V_n]_{kk'} &= \sum_{j=0}^{n-1} (\omega_n^{-jk}/n)(\omega_n^{jk'}) \\ &= \sum_{j=0}^{n-1} \omega_n^{j(k'-k)}/n. \end{aligned}$$

This summation equals 1 if $k' = k$, and it is 0 otherwise by the summation lemma (Lemma 30.6). Note that in order for the summation lemma to apply, $k' - k$ must not be divisible by n . Indeed, it is not, since $-(n-1) \leq k' - k \leq n-1$. ■

With the inverse matrix V_n^{-1} defined, $\text{DFT}_n^{-1}(y)$ is given by

$$\begin{aligned} a_j &= \sum_{k=0}^{n-1} y_k \frac{\omega_n^{-jk}}{n} \\ &= \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj} \end{aligned} \tag{30.11}$$

for $j = 0, 1, \dots, n-1$. By comparing equations (30.8) and (30.11), you can see that if you modify the FFT algorithm to switch the roles of a and y , replace ω_n by ω_n^{-1} , and divide each element of the result by n , you get the inverse DFT (see Exercise 30.2-4). Thus, DFT_n^{-1} is computable in $\Theta(n \lg n)$ time as well.

Thus, the FFT and the inverse FFT provide a way to transform a polynomial of degree-bound n back and forth between its coefficient representation and a point-value representation in only $\Theta(n \lg n)$ time. In the context of polynomial multiplication, we have shown the following about the convolution $a \otimes b$ of vectors a and b :

Theorem 30.8 (Convolution theorem)

For any two vectors a and b of length n , where n is an exact power of 2,

$$a \otimes b = \text{DFT}_{2n}^{-1}(\text{DFT}_{2n}(a) \cdot \text{DFT}_{2n}(b)),$$

where the vectors a and b are padded with 0s to length $2n$ and $\cdot$ denotes the componentwise product of two $2n$ -element vectors. ■

Exercises**30.2-1**

Prove Corollary 30.4.

30.2-2

Compute the DFT of the vector $(0, 1, 2, 3)$.

30.2-3

Do Exercise 30.1-1 by using the $\Theta(n \lg n)$ -time scheme.

30.2-4

Write pseudocode to compute DFT_n^{-1} in $\Theta(n \lg n)$ time.

30.2-5

Describe the generalization of the FFT procedure to the case in which n is an exact power of 3. Give a recurrence for the running time, and solve the recurrence.

★ 30.2-6

Instead of performing an n -element FFT over the field of complex numbers (where n is an exact power of 2), let's use the ring $\mathbb{Z}_m$ of integers modulo m , where $m = 2^{tn/2} + 1$ and t is an arbitrary positive integer. We can use $\omega = 2^t$ instead of ω_n as a principal n th root of unity, modulo m . Prove that the DFT and the inverse DFT are well defined in this system.

30.2-7

Given a list of values $z_0, z_1, \dots, z_{n-1}$ (possibly with repetitions), show how to find the coefficients of a polynomial $P(x)$ of degree-bound $n + 1$ that has zeros only at $z_0, z_1, \dots, z_{n-1}$ (possibly with repetitions). Your procedure should run in $O(n \lg^2 n)$ time. (*Hint:* The polynomial $P(x)$ has a zero at z_j if and only if $P(x)$ is a multiple of $(x - z_j)$.)

★ 30.2-8

The **chirp transform** of a vector $a = (a_0, a_1, \dots, a_{n-1})$ is the vector $y = (y_0, y_1, \dots, y_{n-1})$, where $y_k = \sum_{j=0}^{n-1} a_j z^{kj}$ and z is any complex number. The DFT is therefore a special case of the chirp transform, obtained by taking $z = \omega_n$. Show how to evaluate the chirp transform for any complex number z in $O(n \lg n)$ time. (*Hint:* Use the equation

$$y_k = z^{k^2/2} \sum_{j=0}^{n-1} \left(a_j z^{j^2/2} \right) \left(z^{-(k-j)^2/2} \right)$$

to view the chirp transform as a convolution.)

30.3 FFT circuits

Many of the FFT's applications in signal processing require the utmost speed, and so the FFT is often implemented as a circuit in hardware. The FFT's divide-and-conquer structure enables the circuit to have a parallel structure so that the *depth* of the circuit—the maximum number of computational elements between any output and any input that can reach it—is $\Theta(\lg n)$. Moreover, the structure of the FFT circuit has several interesting mathematical properties, which we won't go into here.

Butterfly operations

Notice that the **for** loop of lines 9–12 of the FFT procedure computes the value $\omega_n^k y_k^{\text{odd}}$ twice per iteration: once in line 10 and once in line 11. A good optimizing compiler produces code that evaluates this *common subexpression* just once, storing its value into a temporary variable, so that lines 10–11 are treated like the three lines

$$\begin{aligned} t &= \omega_n^k y_k^{\text{odd}} \\ y_k &= y_k^{\text{even}} + t \\ y_{k+(n/2)} &= y_k^{\text{even}} - t \end{aligned}$$

This operation, multiplying the twiddle factor $\omega = \omega_n^k$ by y_k^{odd} , storing the product into the temporary variable t , and adding and subtracting t from y_k^{even} , is known as a *butterfly operation*. Figure 30.3 shows it as a circuit, and you can see how it vaguely resembles the shape of a butterfly. (Although less colorfully, it could have been called a “bowtie” operation.)

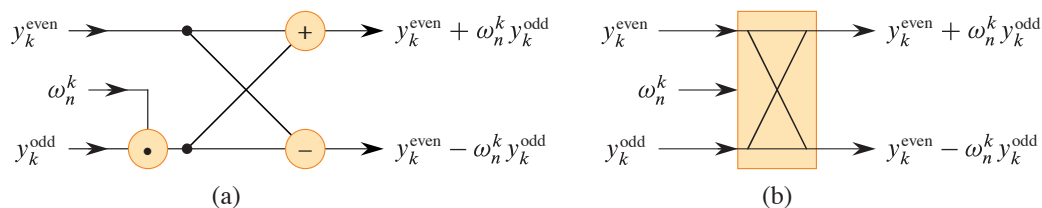


Figure 30.3 A circuit for a butterfly operation. (a) The two input values enter from the left, the twiddle factor ω_n^k is multiplied by y_k^{odd} , and the sum and difference are output on the right. (b) A simplified drawing of a butterfly operation, which we'll use when drawing the parallel FFT circuit.

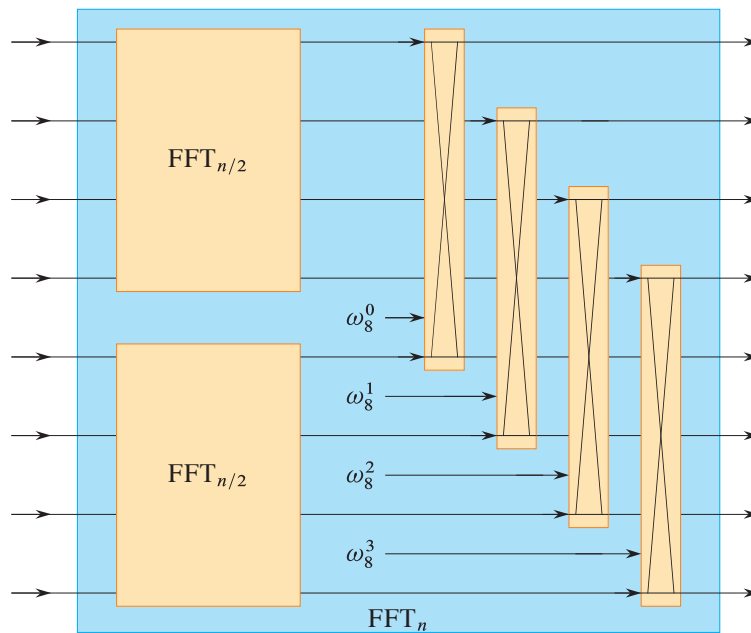


Figure 30.4 The schema for the conquer and combine steps of an n -input, n -output FFT circuit, FFT_n , shown for $n = 8$. Inputs enter from the left, and outputs exit from the right. The input values first go through two $\text{FFT}_{n/2}$ circuits, and then $n/2$ butterfly circuits combine the results. Only the top and bottom wires entering a butterfly interact with it: wires that pass through the middle of a butterfly do not affect that butterfly, nor are their values changed by that butterfly.

Recursive circuit structure

The FFT procedure follows the divide-and-conquer strategy that we first saw in Section 2.3.1:

Divide the n -element input vector into its $n/2$ even-indexed and $n/2$ odd-indexed elements.

Conquer by recursively computing the DFTs of the two subproblems, each of size $n/2$.

Combine by performing $n/2$ butterfly operations. These butterfly operations work with twiddle factors $\omega_n^0, \omega_n^1, \dots, \omega_n^{n/2-1}$.

The circuit schema in Figure 30.4 follows the conquer and combine steps of this pattern for an FFT circuit with n inputs and n outputs, denoted by FFT_n . Each line is a wire that carries a value. Inputs enter from the left, one per wire, and outputs exit from the right. The conquer step runs the inputs through two $\text{FFT}_{n/2}$ circuits, which are also constructed recursively. The values produced by the two $\text{FFT}_{n/2}$ circuits feed into $n/2$ butterfly circuits, with twiddle factors $\omega_n^0, \omega_n^1, \dots, \omega_n^{n/2-1}$,

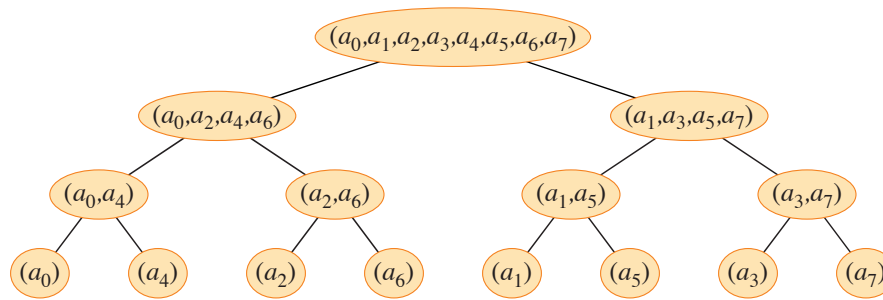


Figure 30.5 The tree of input vectors to the recursive calls of the FFT procedure. The initial invocation is for $n = 8$.

to combine the results. The base case of the recursion occurs when $n = 1$, where the sole output value equals the sole input value. An FFT_1 circuit, therefore, does nothing, and so the smallest nontrivial FFT circuit is FFT_2 , a single butterfly operation whose twiddle factor is $\omega_2^0 = 1$.

Permuting the inputs

How does the divide step enter into the circuit design? Let's examine how input vectors to the various recursive calls of the FFT procedure relate to the original input vector, so that the circuit can emulate the divide step at the start for all levels of recursion. Figure 30.5 arranges the input vectors to the recursive calls in an invocation of FFT in a tree structure, where the initial call is for $n = 8$. The tree has one node for each call of the procedure, labeled by the elements of the initial call as they appear in the corresponding input vector. Each FFT invocation makes two recursive calls, unless it has received a 1-element vector. The first call appears in the left child, and the second call appears in the right child.

Looking at the tree, observe that if you arrange the elements of the initial vector a into the order in which they appear in the leaves, you can trace the execution of the FFT procedure, but bottom up instead of top down. First, take the elements in pairs, compute the DFT of each pair using one butterfly operation, and replace the pair with its DFT. The vector then holds $n/2$ two-element DFTs. Next, take these $n/2$ DFTs in pairs and compute the DFT of the four vector elements they come from by executing two butterfly operations, replacing two two-element DFTs with one four-element DFT. The vector then holds $n/4$ four-element DFTs. Continue in this manner until the vector holds two $(n/2)$ -element DFTs, which $n/2$ butterfly operations combine into the final n -element DFT. In other words, you can start with the elements of the initial vector a , but rearranged as in the leaves of Figure 30.5, and then feed them directly into a circuit that follows the schema in Figure 30.4.

Let's think about the permutation that rearranges the input vector. The order in which the leaves appear in Figure 30.5 is a **bit-reversal permutation**. That is, letting $\text{rev}(k)$ be the $\lg n$ -bit integer formed by reversing the bits of the binary representation of k , then vector element a_k moves to position $\text{rev}(k)$. In Figure 30.5, for example, the leaves appear in the order 0, 4, 2, 6, 1, 5, 3, 7. This sequence in binary is 000, 100, 010, 110, 001, 101, 011, 111, and you can obtain it by reversing the bits of each number in the sequence 0, 1, 2, 3, 4, 6, 7 or, in binary, 000, 001, 010, 011, 100, 101, 110, 111. To see in general that the input vector should be rearranged by a bit-reversal permutation, note that at the top level of the tree, indices whose low-order bit is 0 go into the left subtree and indices whose low-order bit is 1 go into the right subtree. Stripping off the low-order bit at each level, continue this process down the tree, until you get the order given by the bit-reversal permutation at the leaves.

The full FFT circuit

Figure 30.6 depicts the entire circuit for $n = 8$. The circuit begins with a bit-reversal permutation of the inputs, followed by $\lg n$ stages, each stage consisting of $n/2$ butterflies executed in parallel. Assuming that each butterfly circuit has constant depth, the full circuit has depth $\Theta(\lg n)$. The butterfly operations at each level of recursion in the FFT procedure are independent, and so the circuit performs them in parallel. The figure shows wires running from left to right, carrying values through the $\lg n$ stages. For $s = 1, 2, \dots, \lg n$, stage s consists of $n/2^s$ groups of butterflies, with 2^{s-1} butterflies per group. The twiddle factors in stage s are $\omega_m^0, \omega_m^1, \dots, \omega_m^{m/2-1}$, where $m = 2^s$.

Exercises

30.3-1

Show the values on the wires for each butterfly input and output in the FFT circuit of Figure 30.6, given the input vector $(0, 2, 3, -1, 4, 5, 7, 9)$.

30.3-2

Consider an FFT_n circuit, such as in Figure 30.6, with wires $0, 1, \dots, n-1$ (wire j has output y_j) and stages numbered as in the figure. Stage s , for $s = 1, 2, \dots, \lg n$, consists of $n/2^s$ groups of butterflies. Which two wires are inputs and outputs for the j th butterfly circuit in the g th group in stage s ?

30.3-3

Consider a b -bit integer k in the range $0 \leq k < 2^b$. Treating k as a b -element vector over $\{0, 1\}$, describe a $b \times b$ matrix M such that the matrix-vector product Mk is the binary representation of $\text{rev}(k)$.

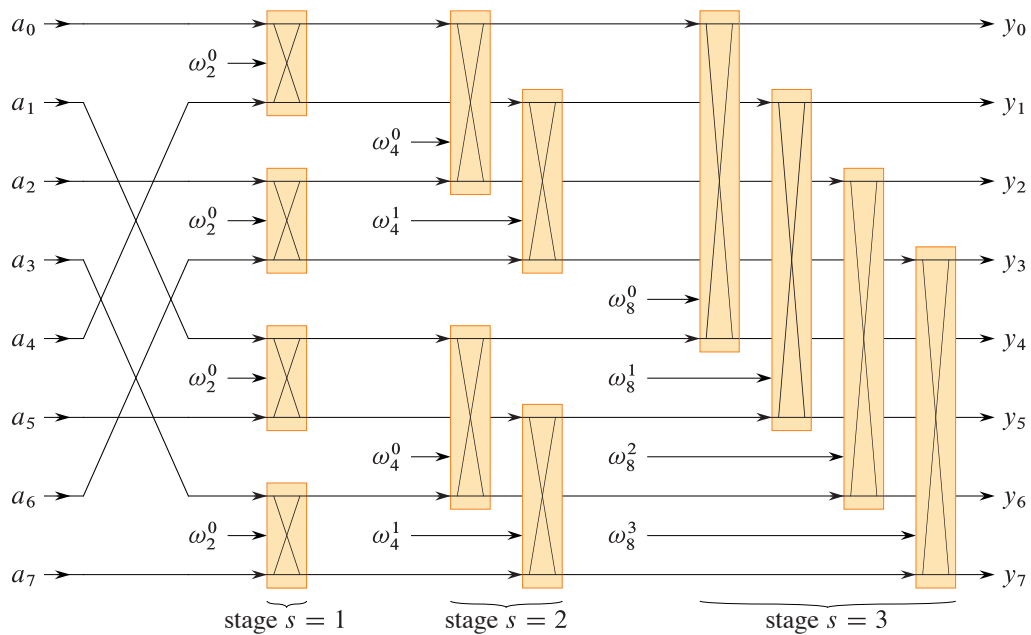


Figure 30.6 A full circuit that computes the FFT in parallel, here shown for $n = 8$ inputs. It has $\lg n$ stages, and each stage comprises $n/2$ butterflies that can operate in parallel. As in Figure 30.4, only the top and bottom wires entering a butterfly interact with it. For example, the top butterfly in stage 2 has inputs and outputs only on wires 0 and 2 (the wires with outputs y_0 and y_2 , respectively). This circuit has depth $\Theta(\lg n)$ and performs $\Theta(n \lg n)$ butterfly operations altogether.

30.3-4

Write pseudocode for the procedure `BIT-REVERSE-PERMUTATION( $a, n$ )`, which performs the bit-reversal permutation on a vector a of length n in-place. Assume that you may call the procedure `BIT-REVERSE-OF( $k, b$ )`, which returns an integer that is the b -bit reversal of the nonnegative integer k , where $0 \leq k < 2^b$.

★ 30.3-5

Suppose that the adders within the butterfly operations of a given FFT circuit sometimes fail in such a manner that they always produce a 0 output, independent of their inputs. In addition, suppose that exactly one adder has failed, but you don't know which one. Describe how you can identify the failed adder by supplying inputs to the overall FFT circuit and observing the outputs. How efficient is your method?